# Data Programming in Python | BAIS:6040

# Homework 3 Solutions

# Module 3 - Python Basics Part 2

## Exercises for For Loops

1. Write a **for** loop that prints all integers from 1 to 100, separated by a tab. Use the **range** function to generate numbers.

In [ ]:
```
# Your answer here
for num in range(1, 101):
    print(num, end="\t")
```

2. Write a **for** loop that prints all characters in the string `s` below, separated by a tab.

In [ ]:
```
s = "abcdefghijklmnopqrstuvwxyz"
```

In [ ]:
```
# Your answer here
for c in s:
    print(c, end="\t")
```

3. Write a **for** loop that prints the same as above, but this time using the index of `s`.

In [ ]:
```
# Your answer here
for i in range(len(s)):
    c = s[i]
    print(c, end="\t")
```

4. Write a **for** loop that prints the same as above, except that the letters at odd index positions are printed in upper case, while the other letters are printed as they are.

In [ ]:
```
# Your answer here
for i in range(len(s)):
    c = s[i]
    if i % 2 == 1:
        print(c.upper(), end="\t")
    else:
        print(c, end="\t")
```

5. Write a **for** loop that prints "NAME is SEX and aged AGE." for each tuple in `employees`, separated by a new line, where NAME is the first value in each tuple, SEX the second, and

AGE the third. For example, the first line in the output must look as follows:

Alice is female and aged 30.

In [ ]:
```
employees = [("Alice", 30, "female"), ("Bob", 25, "male"), ("Tom", 34, "male")]
```

In [ ]:
```
# Your answer here
for name, age, sex in employees:
    print("{} is {} and aged {}.".format(name, sex, age))
```

# Exercises for While Loops

1. Write a `while` loop to copy the strings `'orange'` of the list `squares` to the list `new_squares`. Stop and exit the loop if the value on the list is not `'orange'`:

In [ ]:
```
squares = ['orange', 'orange', 'orange', 'red', 'blue ', 'orange']
new_squares = []
```

In [ ]:
```
# Your answer here
i = 0

while squares[i] == "orange":
    new_squares.append(squares[i])
    i += 1
print(new_squares)
```

2. Write a `while` loop to print the first n natural numbers, where n is an integer that user inputs. hint: use the input() function to prompts user to enter a number.

In [ ]:
```
# Your answer here
n = int(input("enter a number: "))
i = 1

while i <= n:
    print(i)
    i += 1
```

3. Find the sum of even numbers between 0 to 100 using a `while` loop.

In [ ]:
```
# Your answer here
i = 0
total = 0

while i <= 100:
    if i % 2 == 0:
        total += i
        print(i)
    i += 1
print(total)
```

In [ ]:
```
i = 0
sum_even = 0
```

```python
while i <= 100:
    sum_even += i
    i += 2
print(sum_even)
```

# Exercises for List Comprehension

1. Not using and using list comprehension, create a list `l2` of squares of the numbers in `l1`.

In [ ]:
```python
l1 = [1, 2, 3, 4, 5]
```

In [ ]:
```python
# Your answer here NOT using list comprehension
l2 = []

for num in l1:
    l2.append(num ** 2)

l2
```

In [ ]:
```python
# Your answer here USING list comprehension
l2 = [num ** 2 for num in l1]
l2
```

2. Not using and using list comprehension, create a list `l3` of lists, each of which contains the square and cube of each number in `l1`. For example, the last element in `l3` is [25, 125].

In [ ]:
```python
# Your answer here NOT using list comprehension
l3 = []

for num in l1:
    l3.append([num ** 2, num ** 3])

l3
```

In [ ]:
```python
# Your answer here using list comprehension
l3 = [[num ** 2, num ** 3] for num in l1]
l3
```

3. Using list comprehension, create a list `names2` of the names in `names` in lower case.

In [ ]:
```python
names = ["Alice", "Bob", "Tom"]
```

In [ ]:
```python
names2 = []
for name in names:
    names2.append(name.lower())
names2
```

In [ ]:
```
# Your answer here
names2 = [name.lower() for name in names]
names2
```

4. Using list comprehension, create a list `names3` of tuples, each of which contains each name in `names` and the length of the name. For example, the first element in `names` is ('Alice', 5).

In [ ]:
```
names3 = []
for name in names:
    names3.append((name, len(name)))
names3
```

In [ ]:
```
[(n, len(n)) for n in names]
```

In [ ]:
```
# Your answer here
names3 = [(name, len(name)) for name in names]
names3
```

5. Given a matrix (a list of lists) of numbers, use a list comprehension to create a new flattened list that contains only the even numbers from the matrix.

In [ ]:
```
matrix = [
    [1, 2, 3, 4],
    [4, 5, 6, 8],
    [7, 8, 9, 10]
]
```

In [ ]:
```
# Your answer here
flattened_list = [num for row in matrix for num in row if num % 2 == 0]
flattened_list
```

In [ ]:
```
flattened_list = []
for row in matrix:
    for num in row:
        if num %2 ==0:
            flattened_list.append(num)

flattened_list
```

# Exercises for Loop Control

1. Write a **for** loop that prints all names in `names` except for "*Mary*". (Use **continue**.)

In [14]:
```
names = ["Alice", "Bob", "Mary", "Sam", "Sarah", "Tom"]
```

In [ ]:
```
# Your answer here
for name in names:
    if name == "Mary":
```

```
        continue
    print(name)
```

2. Write a **for** loop that prints the names in `names` from the beginning but stops right after it has printed "*Mary*".

In [16]:
```
for name in names:
    print(name)

    if name == "Mary":
        continue
```

```
Alice
Bob
Mary
Sam
Sarah
Tom
```

In [ ]:
```
# Your answer here
for name in names:
    print(name)

    if name == "Mary":
        break
```

3. Write a **for** loop that adds all numbers in `l` except for multiples of 5. Print the final sum after the **for** loop. (Use **continue**.)

In [ ]:
```
l = range(1, 1001)
```

In [ ]:
```
# Your answer here
sum1 = 0

for num in l:
    if num % 5 == 0:
        continue
    sum1 += num

sum1
```

4. Write a **for** loop that adds numbers in `l` from the beginning but stops when the intermediate sum exceeds 1000, printing the current sum and the number last added.

In [ ]:
```
# Your answer here
sum1 = 0

for num in l:
    sum1 += num

    if sum1 > 1000:
        print(sum1, num)
        break
```